

Multi-Rail Fintech Blueprints | Technical White Paper & Structural
Blueprint | Paper II

THE SYSTEM CONNECTOR ENGINE: ROUTING INTELLIGENCE, CROSS-RAIL STATE MANAGEMENT, AND COMPLIANCE SYNCHRONIZATION IN THE MULTI-RAIL RWA STACK

*A Full Internal Engineering Specification of the Central Middleware Layer in the
Multi-Rail Architecture*

Author: Leon Calvin Long II

Date: July 13, 2026

Classification: Technical White Paper & Structural Blueprint

Series: Multi-Rail Fintech Blueprints — Paper II of an ongoing series

*Prerequisite Reading: Paper I — The Architecture of Compliant, Cross-Chain RWA Tokenization in the
2026 Post-Silo Economy*

1. Abstract

Abstract

Paper I established the foundational three-tiered architecture of the multi-rail RWA stack: the **Velocity Layer** (XRPL, Stellar, Shibarium), the **Trust Layer** (MPC plus state-vector anchoring), and the **Compliance/Identity Layer** (ZK proofs plus Travel Rule). Within that architecture, the **System Connector Engine (SCE)** appeared as a single labeled box — the central middleware sitting between the Compliance/Identity Layer and the three settlement rails. Paper I named it. Paper II engineers it.

The SCE is not a simple message router. It is an active, stateful middleware layer responsible for six distinct operational functions: (1) asset-profile-aware rail selection, (2) cross-rail state vector propagation, (3) compliance envelope synchronization across rail boundaries, (4) oracle arbitration for pricing and sanctions data, (5) error detection and atomic rollback, and (6) the embedded protocol fee stream. Each function is interdependent; none can be delegated to the rails themselves or to the Compliance Layer without collapsing the architectural guarantees that Paper I established.

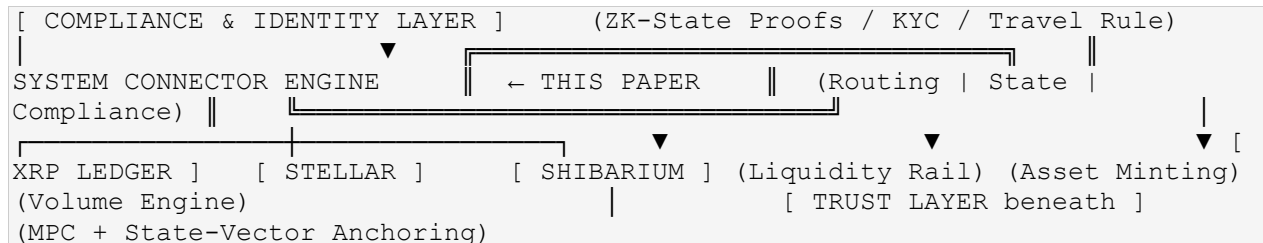
Without a correctly architected SCE, the three-rail Velocity Layer collapses into three disconnected silos — reproducing exactly the legacy fragmentation the blueprint universe was designed to eliminate. This paper provides the complete internal blueprint of the SCE: component diagrams, data schemas, routing algorithms, state machine specifications,

compliance handshake protocols, and operational deployment requirements sufficient for production implementation.

2. Recap: Where the SCE Sits in the Stack

2.1 The Paper I Architecture, Annotated

Paper I presented a three-tiered vertical stack. For the reader's reference, that architecture is reproduced below with the SCE box explicitly annotated as the subject of this paper:



In Paper I, the SCE was described as the routing middleware that translates a compliant asset transfer instruction into a rail-specific transaction. That description was accurate but incomplete. The SCE is not merely a translator. It is the owner of the canonical cross-rail state, the enforcer of compliance continuity across settlement legs, and the mechanism through which the protocol's revenue model is executed. Every architectural guarantee that Paper I made about the multi-rail stack is implemented by the SCE or depends on the SCE's correct operation.

2.2 The SCE's Three Primary Contracts

The SCE operates under three binding architectural contracts — two external (with the layers above and below it) and one internal.

Contract 1 — Upward, with the Compliance/Identity Layer: Accept a ZK-verified asset transfer instruction and its compliance attestation. Never dispatch a transaction to any rail unless the compliance attestation is cryptographically valid, within its validity window, and has been logged to the State Vector. This contract is non-negotiable. Any SCE implementation that permits rail dispatch without a valid attestation is architecturally non-compliant with the blueprint.

Contract 2 — Downward, with the Velocity Layer: Dispatch every transaction to exactly one rail per settlement leg, selected by the Routing Matrix. Monitor finality on the selected rail. Propagate the confirmed finality state upward to the State Vector Manager. Never declare settlement without cryptographic confirmation from the target rail.

Contract 3 — Internal: Maintain a consistent, append-only cross-rail state record that no individual rail can provide. The individual rails — XRPL, Stellar, Shibaumi — have no visibility into each other's transaction sets. The SCE is the only component in the stack with a unified view of every in-flight and settled transaction across all three rails simultaneously. This internal contract is what makes multi-leg, cross-rail settlement atomically safe.

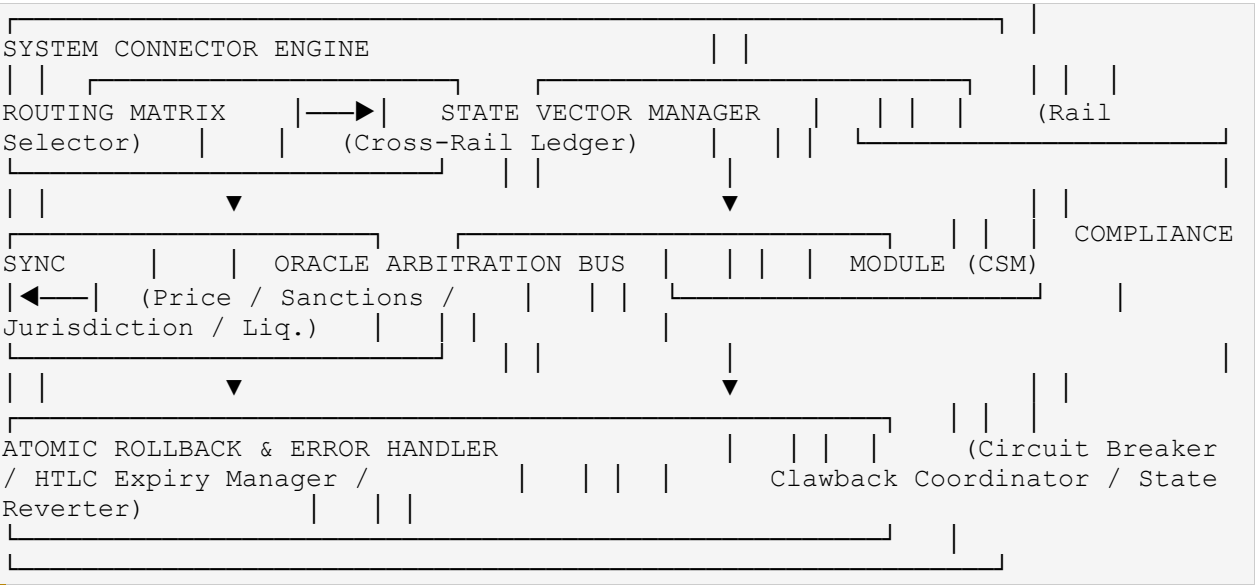
2.3 The SCE Is Stateful by Design

A critical architectural decision that must be understood before reviewing the sub-module specifications: the SCE is explicitly **not** a stateless API gateway. It is not a proxy, a load balancer, or a routing table. It is a stateful middleware system that owns and maintains the canonical settlement record. Engineers accustomed to designing stateless microservices must treat the SCE's state store as a first-class system component, not an afterthought. The operational complexity of running the SCE correctly is almost entirely a function of managing that state safely across a distributed deployment.

3. Internal Architecture of the SCE

3.1 Component Overview

The SCE is composed of five internal sub-modules, each with a distinct responsibility and a defined interface with the others. The component diagram below shows the sub-modules and their primary data flows:



3.2 Sub-Module Descriptions

3.2.1 — Routing Matrix

The **Routing Matrix** is the asset-profile-aware rail selector. It receives an incoming asset transfer instruction from the Compliance/Identity Layer (post-attestation) and determines which rail — XRPL, Stellar, or Shibarium — is the optimal settlement destination for that specific transaction at that specific moment. Inputs include: asset class identifier, transaction value in USD, counterparty jurisdictions, current liquidity availability across all three rails (sourced from the Oracle Arbitration Bus), and throughput priority. Output is a rail assignment with an ordered fallback priority list. The Routing Matrix never dispatches the transaction itself; it produces a routing decision record that the State Vector Manager logs before the transaction is dispatched.

3.2.2 — State Vector Manager (SVM)

The **State Vector Manager** is the SCE's authoritative ledger. It maintains the canonical cross-rail state for every transaction the SCE has touched. Every state transition — from receipt of a compliance attestation through final settlement — is written to the SVM's append-only state store. The SVM is the only sub-module with write access to the canonical state store. All other sub-modules report state change events to the SVM; none write directly. This single-writer design eliminates race conditions on the state record. The SVM also runs the per-rail Finality Monitors and calculates the protocol fee at the `CROSS_RAIL_SETTLED` state transition.

3.2.3 — Compliance Sync Module (CSM)

The **Compliance Sync Module** is the SCE's compliance enforcement arm. It receives compliance attestations from the Compliance/Identity Layer, verifies their validity window, packages them into rail-specific compliance envelopes, manages Travel Rule data package dispatch and acknowledgment, monitors compliance state through the settlement lifecycle, and coordinates clawback or HTLC abandonment if a compliance breach occurs mid-flight. The CSM communicates the compliance status of every transaction to the SVM at each state transition.

3.2.4 — Oracle Arbitration Bus (OAB)

The **Oracle Arbitration Bus** aggregates external data feeds required by other SCE sub-modules: asset price data (for fee computation and routing fitness scoring), sanctions and compliance list data (for real-time screening), jurisdictional rule updates (Travel Rule thresholds, eligible investor criteria), and real-time liquidity depth readings from all three rail pools. The OAB applies a weighted median arbitration algorithm to eliminate single-oracle dependency. It is the only SCE sub-module with persistent external network connections; all other sub-modules consume data exclusively through the OAB's internal API.

3.2.5 — Atomic Rollback & Error Handler

The **Atomic Rollback & Error Handler** monitors all in-flight transactions across all rails. On detection of a failure — rail submission rejection, finality timeout, compliance breach, or oracle quorum failure — it determines the appropriate rollback mechanism (HTLC expiry invocation, Stellar clawback trigger, or state vector reversion) and executes it atomically. It also implements the Circuit Breaker pattern for per-rail fault isolation, preventing a single degraded rail from poisoning the entire SCE pipeline.

4. The Routing Matrix: Rail Selection Algorithm

4.1 The Routing Problem

The Routing Matrix must answer a deceptively complex question for every incoming transaction: given this specific asset, this specific value, these specific counterparty jurisdictions, and current rail conditions, which rail produces the optimal settlement outcome? The answer is not static. The same asset type might route to Stellar at 10:00 AM and to XRPL at 10:05 AM if Stellar's liquidity pool has been partially drained by a large preceding transaction. The Routing Matrix must be continuously informed, deterministically scored, and operationally transparent — every routing decision must be logged with the full scoring record so that post-hoc audits can reconstruct exactly why a given transaction was assigned to a given rail.

4.2 The Asset Profile Vector

Every incoming transaction is characterized by an **Asset Profile Vector**, defined as:

$$P^{\rightarrow} = (C, V, J, L, T)$$

Where each component is defined as follows:

- **C** — Asset Class Score (integer 0–10): Reflects the regulatory complexity and institutional grade of the asset. A score of 10 represents sovereign bonds or real estate; a score of 0 represents community or memetic tokens with no regulatory registration. This score is defined in the asset's registry entry, not computed at runtime.
- **V** — Transaction Value in USD (float): The gross transaction value in USD at the time of routing, computed using the OAB's current price oracle reading for the asset.
- **J** — Jurisdictional Complexity Score (integer 0–10): Computed as the count of distinct regulatory jurisdictions involved in the transaction multiplied by a configurable per-jurisdiction regulatory

stringency weight. A domestic USD stablecoin transfer scores near 1; a cross-border RWA transfer touching the EU, Singapore, and UAE simultaneously may score 8 or 9.

- **L** — Required Liquidity Depth in USD (float): The minimum USD-equivalent liquidity that must be available in the destination rail's settlement pool at the time of dispatch. Derived from V with a configurable multiplier per asset class.
- **T** — Throughput Priority (float 0.0–1.0): A value of 0.0 indicates that finality certainty is the priority (institutional settlement); a value of 1.0 indicates that speed is the priority (retail micro-transaction). This value is set by the originating institution in the transfer instruction.

4.3 The Rail Fitness Score

The Routing Matrix computes a **Rail Fitness Score** for each candidate rail $r \in \{XRPL, Stellar, Shibarium\}$:

$$F(r, P^*) = w_1 \cdot Compliance(r, C, J) + w_2 \cdot Liquidity(r, L) + w_3 \cdot Throughput(r, T) - w_4 \cdot Cost(r, V)$$

Where $w_1 + w_2 + w_3 + w_4 = 1$ and the weights are configurable per deployment via the signed configuration manifest. The component functions are defined as:

- **Compliance(r, C, J)**: A lookup against the rail's compliance ceiling (see Table 4.1) normalized to $[0, 1]$. A rail whose compliance ceiling is below the asset's C score returns 0, immediately disqualifying it from routing regardless of other scores.
- **Liquidity(r, L)**: The ratio of current pool depth on rail r to the required liquidity L , capped at 1.0. If pool depth falls below L , this component returns 0 and the rail is disqualified.
- **Throughput(r, T)**: A normalized score based on the rail's TPS capacity weighted by the transaction's T value. High- T transactions score Shibarium highest; low- T transactions weight Stellar and XRPL's deterministic finality more heavily.
- **Cost(r, V)**: The estimated transaction cost on rail r for value V , normalized to $[0, 1]$ and subtracted — lower cost rails produce higher fitness scores, all else being equal.

4.4 Rail Fitness Reference Table

Rail	Compliance Ceiling	Liquidity Depth	Throughput (TPS)	Base Cost	Best-Fit Asset Class
Stellar	10 / 10	Medium	~1,000 TPS	< \$0.001	RWA, Securities, Regulated Stablecoins
XRPL	8 / 10	High (AMM + DEX)	~1,500 TPS	< \$0.001	Cross-border Settlement, FX Corridors
Shibarium	5 / 10	Very High (ERC-20 pools)	~100,000 TPS	Low (L2)	Community Tokens, Retail Fractionalization

Table 4.1 — Rail Fitness Reference Data. Compliance ceiling reflects the maximum Asset Class Score (C) the rail can handle given its current regulatory registration profile. These values are deployment-configurable and should be reviewed whenever a rail's regulatory status changes.

⚠ Deployment Warning

A rail whose compliance ceiling is strictly less than the asset's C score must be treated as disqualified — not merely penalized. Routing a C=9 sovereign bond transaction to Shibarium (ceiling 5) is a compliance architecture violation, not a suboptimal routing choice. The Routing Matrix must implement this disqualification as a hard gate, not a weighted factor.

4.5 Fallback Routing and Re-Scoring

The Routing Matrix always produces an ordered list of rail assignments, not a single selection. If the primary rail becomes unavailable after the routing decision is made — or if its real-time liquidity reading from the OAB falls below the required threshold L at the moment of dispatch — the SCE automatically re-scores against the next candidate on the fallback list and reroutes without requiring operator

intervention. Every rerouting event is written to the State Vector as a routing amendment record, preserving the full decision trail.

Fallback routing is not a degraded mode. It is an expected operational condition. Rail liquidity conditions fluctuate continuously, and large institutional transactions will regularly exhaust primary pool depth.

The Routing Matrix's fallback mechanism is the operational answer to that reality.

4.6 Routing Decision Record — JSON Blueprint

```
{  "routing_decision": {    "transaction_id": "SCE-2026-07-TX-00412",    "asset_profile": {      "asset_class_score": 9,      "value_usd": 250000.00,      "jurisdictional_complexity": 7,      "required_liquidity_usd": 100000,      "throughput_priority": 0.1    },    "rail_fitness_scores": {      "stellar": 0.87,      "xrpl": 0.72,      "shibarium": 0.00    },    "disqualified_rails": {      "shibarium": 0.00    },    "compliance_ceiling_breach": "asset_class_score 9 exceeds ceiling 5",    "selected_rail": "stellar",    "fallback_priority": ["xrpl"],    "routing_timestamp": "2026-07-13T19:42:00Z",    "routing_engine_version": "SCE-v2.4.1",    "weight_config": {      "w1_compliance": 0.40,      "w2_liquidity": 0.30,      "w3_throughput": 0.15,      "w4_cost": 0.15    },    "oracle_snapshot_id": "OAB-SNAP-20260713T194200Z"  } }
```

Note the `oracle_snapshot_id` field. Every routing decision is pinned to a specific oracle data snapshot, ensuring that the liquidity and pricing data used for the routing computation can be independently reconstructed and audited. The snapshot ID references a record in the OAB's internal log.

5. The State Vector Manager: Cross-Rail State Ledger

5.1 The State Problem

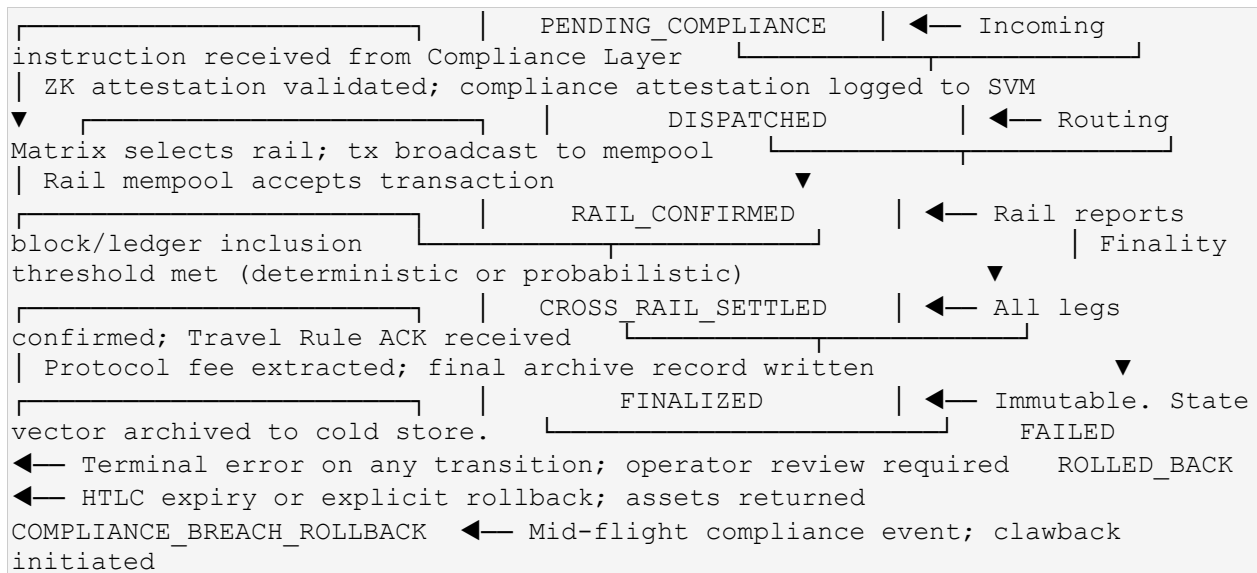
Each rail in the Velocity Layer has its own finality model, and none of them are compatible with each other. XRPL achieves deterministic finality in approximately 3–4 seconds, confirmed by ledger close.

Stellar achieves deterministic finality in approximately 5 seconds, confirmed by ledger sequence number. Shibarium, as an EVM-compatible Layer 2, uses probabilistic finality — each additional block confirmation deepens the security of the settlement, but no single block provides the same finality guarantee as a closed XRPL or Stellar ledger. The default Shibarium finality threshold in the SCE is 12 block confirmations, configurable per deployment.

These three finality models are incommensurable at the individual rail level. No single rail can tell you whether a multi-leg transaction that touches both Stellar (for the asset leg) and XRPL (for the cash leg) is settled. Only the SCE — through the State Vector Manager — can answer that question, because only the SVM monitors both rails simultaneously and maintains a unified state record that integrates both finality confirmations into a single settlement event.

5.2 The Transaction State Machine

Every transaction in the SCE progresses through a defined state machine with five primary states and two terminal error states:



Architectural Note

The FINALIZED state is immutable by design. No operator action, configuration change, or system event can modify a FINALIZED state vector. If a post-finalization legal action (court order, regulatory recall) requires a correction to a finalized transaction, that correction is implemented as a new, separate transaction in the forward direction — never as a modification to the original state record.

5.3 The State Vector Schema — JSON Blueprint

```
{  "state_vector": {    "transaction_id": "SCE-2026-07-TX-00412",    "current_state": "CROSS_RAIL_SETTLED",    "state_history": [      {        "state": "PENDING_COMPLIANCE",        "timestamp": "2026-07-13T19:41:58Z",        "actor": "compliance_layer",        "attestation_hash": "sha512:a9f3c2e8d47b...",        "state": "DISPATCHED",        "timestamp": "2026-07-13T19:42:00Z",        "actor": "routing_matrix",        "rail": "stellar",        "routing_decision_ref": "SCE-2026-07-TX-00412-RD"      },      {        "state": "RAIL_CONFIRMED",        "timestamp": "2026-07-13T19:42:06Z",        "actor": "stellar_finality_monitor",        "ledger_seq": 49203871,        "tx_hash": "stellar:8f4c29a1..."      },      {        "state": "CROSS_RAIL_SETTLED",        "timestamp": "2026-07-13T19:42:07Z",        "actor": "state_vector_manager",        "travel_rule_ack_ref": "TR-ACK-00412"      }    ],    "legs": [      {        "leg_id": "LEG-001",        "leg_type": "asset_delivery",        "rail": "stellar",        "status": "FINALIZED",        "finality_ledger": 49203871,        "finality_confirmed_at": "2026-07-13T19:42:06Z"      },      {        "leg_id": "LEG-002",        "leg_type": "cash_settlement",        "rail": "xrpl",        "status": "FINALIZED",        "finality_ledger": 88104522,        "finality_confirmed_at": "2026-07-13T19:42:09Z"      }    ],    "compliance_attestation_hash": "sha512:a9f3c2e8d47b...",    "travel_rule_ack": true,    "travel_rule_ack_timestamp": "2026-07-13T19:42:03Z",    "protocol_fee_usd": 25.00,    "protocol_fee_rate": 0.0001,    "protocol_fee_tx_ref": "SCE-FEE-00412",    "archive_timestamp": null,    "schema_version": "sv-3.1"  } }
```

5.4 Finality Monitoring Architecture

The SVM runs a dedicated **Finality Monitor** process for each rail. Each monitor is a polling agent that queries its rail's API at a configurable interval (default: 500ms for XRPL and Stellar, 2s for Shibarium) and checks for the inclusion and finality status of every dispatched transaction in the DISPATCHED state.

For **XRPL**: the Finality Monitor subscribes to the XRPL WebSocket API and listens for the `ledgerClosed` event on the ledger containing the transaction. One close event with the transaction present and validated is sufficient for finality.

For **Stellar**: the Finality Monitor polls the Stellar Horizon API for the transaction's ledger sequence number. One ledger close with the transaction present is sufficient for finality.

For **Shibarium**: the Finality Monitor polls the Shibarium RPC endpoint for the transaction receipt and tracks the current block height minus the transaction block height. State advancement to `RAIL_CONFIRMED` is gated behind the configured block depth threshold (default: 12). Operators running high-value institutional transactions on Shibarium should consider raising this threshold to 32 or higher.

The state store is an append-only ledger. No record is ever deleted, modified, or compacted out of the primary state store. Historical state vectors are archived to cold storage after 90 days (configurable) but remain immutably retrievable. This append-only design is the SCE's contribution to the audit lineage required by MiCA, DORA, and the Travel Rule frameworks cited in Paper I's compliance analysis.

6. The Compliance Sync Module: Cross-Rail Compliance Handshake

6.1 The CSM's Role in the Architecture

Paper I's Compliance/Identity Layer (Section 4) issues a ZK proof attesting that the originating party has satisfied KYC, AML, and jurisdictional eligibility requirements. That proof is valid for a configurable time window. The critical problem the CSM solves: what happens when a transaction takes longer to settle than the proof's validity window? What happens when a sanctions list is updated between dispatch and finality? What happens when a multi-leg transaction's first leg clears compliance but its second leg encounters a newly flagged counterparty address?

These are not edge cases. In a production multi-rail system processing institutional volume, they are guaranteed to occur. The CSM is the architectural answer to all three scenarios.

6.2 Three Compliance Obligations the CSM Enforces

6.2.1 — ZK Attestation Relay and Window Monitoring

At the `PENDING_COMPLIANCE` state transition, the CSM receives the ZK proof from the Compliance/Identity Layer, hashes it (SHA-512), and stores the hash in the State Vector. Before any rail dispatch, the CSM performs a validity window check: is the current timestamp within the proof's `valid_until` boundary? If not, the transaction is held in `PENDING_COMPLIANCE` and the Compliance Layer is signaled to issue a fresh attestation. Dispatch is blocked until a valid attestation is present. This check is re-executed at every state transition — a proof that is valid at `PENDING_COMPLIANCE` may expire before `DISPATCHED` if processing is delayed by a compliance queue backlog.

6.2.2 — Travel Rule Envelope Management

For transactions exceeding the statutory threshold (default: \$3,000 USD, configurable to match jurisdiction-specific requirements), the CSM assembles a **Travel Rule Envelope** — the off-chain data package required by FATF Recommendation 16 and its national implementations. The CSM dispatches this envelope to the beneficiary institution's registered VASP gateway simultaneously with the on-chain transaction broadcast. This simultaneous dispatch is intentional: the on-chain and off-chain legs of the Travel Rule obligation must be co-initiated.

The CSM blocks state advancement from `RAIL_CONFIRMED` to `CROSS_RAIL_SETTLED` until it receives a cryptographically signed acknowledgment from the beneficiary VASP's gateway endpoint. If the ACK is not received within the configured timeout (default: 30 seconds), the CSM enters a retry loop. If the retry limit is exhausted, the transaction is flagged for operator review and held in a `TRAVEL_RULE_PENDING` state — it is neither rolled back nor advanced.

 **Compliance Architecture Warning**

The Travel Rule ACK is not optional for transactions above threshold. An SCE deployment that advances state to FINALIZED without a valid Travel Rule ACK is operating in regulatory violation, regardless of the on-chain settlement status. The CSM's ACK gate is a hard blocking condition, not a logging event.

6.2.3 — Clawback Coordination

If a regulatory trigger arrives after dispatch but before FINALIZED — specifically, a sanctions list update that newly designates a transaction counterparty, or a court-ordered freeze — the CSM evaluates the current state and the technical feasibility of clawback on the target rail:

- For **Stellar legs**: the CSM invokes the Stellar clawback architecture described in Paper I, Section 3.1, signaling the issuing account's clawback flag and instructing the MPC signing nodes to co-sign the clawback transaction. This is only feasible if the asset's `ALLOW_CLAWBACK` flag was set at issuance — a deployment requirement the CSM verifies at asset registration time, not at clawback time.
- For **XRPL legs**: the CSM invokes the HTLC timeout mechanism (Paper I, Section 3.2). If the HTLC has not yet been claimed by the beneficiary, the CSM signals the Atomic Rollback Handler to abandon the HTLC, which returns the escrowed assets to the originator when the timelock expires.
- For **Shibarium legs**: clawback feasibility depends on whether the asset contract implements a clawback interface. For assets that do not, the CSM marks the state as `COMPLIANCE_BREACH_PENDING_MANUAL` and generates an operator alert. This is a known limitation of EVM L2 settlement for highly regulated assets and is one reason the Routing Matrix assigns a compliance ceiling of 5/10 to Shibarium.

6.3 Travel Rule Envelope — JSON Blueprint

```
{
  "travel_rule_envelope": {
    "transaction_id": "SCE-2026-07-TX-00412",
    "threshold_usd": 3000,
    "transaction_value_usd": 250000.00,
    "travel_rule_required": true,
    "protocol_version": "ivms101-2.0",
    "originator": {
      "lei": "254900OPPU84GM83MG36",
      "entity_name": "[ENCRYPTED:aes256-gcm]",
      "account_reference": "[ENCRYPTED:aes256-gcm]",
      "zk_attestation_hash": "sha512:a9f3c2e8d47b...",
      "jurisdiction": "US",
      "account_type": "institutional"
    },
    "beneficiary_institution": {
      "lei": "529900T8BM49AURSDO55",
      "vasp_id": "did:web:beneficiary-vasp.example.com",
      "gateway_endpoint": "https://tr.beneficiary-vasp.example.com/inbound",
      "jurisdiction": "SG"
    },
    "envelope_dispatch_timestamp": "2026-07-13T19:42:00Z",
    "ack_received": true,
    "ack_timestamp": "2026-07-13T19:42:03Z",
    "ack_signature": "ed25519:7f4a91b3...",
    "ack_vasp_id": "did:web:beneficiary-vasp.example.com",
    "retry_count": 0,
    "envelope_hash": "sha256:c3f8a2..."
  }
}
```

6.4 Cross-Rail Compliance Consistency

When a transaction spans two rails — as in the example above, where LEG-001 settles on Stellar and LEG-002 settles on XRPL — the CSM maintains independent compliance state tracking for each leg. Each leg has its own RAIL_CONFIRMED gate and its own finality monitor report. The overall transaction state does not advance to CROSS_RAIL_SETTLED until every individual leg has cleared both technical finality and compliance confirmation independently. A transaction cannot be half-settled from the SCE's perspective. Either all legs are confirmed and compliant, or the rollback handler is invoked for the entire transaction.

7. The Oracle Arbitration Bus

7.1 Why Oracles Are a Systemic Risk

A single manipulated or offline oracle can corrupt every downstream function that depends on it: routing fitness scores are wrong, fees are computed incorrectly, sanctions screens miss newly listed entities, and liquidity depth readings route transactions into insufficient pools. In a traditional single-oracle architecture, oracle failure is a single point of failure for the entire settlement pipeline. The SCE's

Oracle Arbitration Bus eliminates this risk by requiring a quorum of independent oracle sources for every data type it consumes, applying a weighted median aggregation algorithm that is robust to individual oracle manipulation or failure.

7.2 Oracle Data Types

The OAB aggregates four categories of external data, each with a minimum of three independent sources:

1. **Price Oracles:** Real-time asset valuations in USD. Used by the Routing Matrix for fitness scoring (value V), by the SVM for protocol fee calculation, and by the CSM for Travel Rule threshold evaluation. Refresh interval: configurable, default every 30 seconds.
2. **Sanctions & Compliance Oracles:** Real-time updates from OFAC SDN list, UN Consolidated Sanctions list, EU Consolidated Sanctions list, and HMT UK Sanctions list. These are distributed push feeds with a default refresh interval of 15 minutes. The OAB maintains a local sanctions cache and applies delta updates on each refresh cycle.
3. **Jurisdictional Rule Oracles:** Regulatory threshold updates pushed from licensed compliance data providers — Travel Rule amount triggers, eligible investor list changes, holding period modifications, and new jurisdiction registrations. These events are infrequent but high-impact; the OAB processes them with elevated priority and immediately invalidates any cached routing decisions that depend on the changed rule.
4. **Liquidity Depth Oracles:** Real-time pool depth readings from XRPL AMM endpoints, Stellar DEX order book APIs, and Shibarium liquidity pool contracts. Consumed by the Routing Matrix at routing decision time to populate the L component of the Asset Profile Vector. Refresh interval: configurable, default every 10 seconds. For high-volume deployments, sub-second refresh via WebSocket subscriptions is recommended.

7.3 Weighted Median Arbitration Algorithm

Given n oracle readings $\{o_1, o_2, \dots, o_n\}$ each with confidence weight w_i , the OAB computes the aggregated value as the weighted median:

$$\hat{O} = \text{WeightedMedian}(\{(o_i, w_i)\})$$

The weighted median is preferred over the weighted mean because it is resistant to outlier manipulation: a single compromised oracle reporting an extreme value does not shift the aggregated result, whereas a weighted mean can be moved arbitrarily far by a single sufficiently extreme reading. Confidence weights are assigned at oracle registration time based on the source's historical accuracy, uptime, and institutional backing. They are updated periodically by the operator through the signed configuration manifest.

Any oracle reading deviating more than δ standard deviations from $\hat{O}$ (default: $\delta = 2.5\sigma$) is flagged as anomalous and excluded from the aggregation for the current cycle. The excluded reading is logged with the oracle source ID, the deviation magnitude, the timestamp, and the current cycle's aggregated value.

7.4 Oracle Anomaly Response Protocol

When an anomalous oracle reading is detected, the OAB executes the following response in sequence:

5. Log the anomalous reading with source ID, deviation magnitude $(|o_i - \hat{O}|/\sigma)$, and cycle timestamp to the OAB anomaly ledger.
6. Exclude the reading from the current cycle's weighted median computation and flag the source as DEGRADED.
7. Alert the operator monitoring dashboard with the source ID, deviation magnitude, and current DEGRADED status.
8. If fewer than $\lceil n/2 \rceil$ sources remain valid for a given data type after exclusions, declare ORACLE_QUORUM_FAILURE for that data type. Halt all new transaction dispatch that depends on that data type. Existing in-flight transactions that have already been dispatched and have clean compliance state are allowed to progress to finalization using their pinned oracle snapshot — they do not rely on live oracle readings after dispatch.

Deployment Warning

Never operate the OAB with fewer than 3 independent oracle sources per data type. With fewer than 3 sources, the weighted median provides no manipulation resistance — a single bad actor controlling one of two sources can shift the aggregated value to any desired reading. The minimum of 3 sources is not a suggestion; it is a security requirement of the OAB architecture.

7.5 Oracle Reading Schema — JSON Blueprint

```
{  "oracle_reading": {    "oracle_id": "ORB-PRICE-FEED-003",    "data_type": "price",    "asset": "XAU/USD",    "value": 3412.55,    "confidence_weight": 0.91,    "source_endpoint": "https://oracle3.example-price-feed.com/xau",    "timestamp": "2026-07-13T19:41:55Z",    "signature": "ed25519:3c8f20a7...",    "anomaly_flagged": false,    "deviation_from_median_sigma": 0.34,    "oracle_status": "ACTIVE",    "cycle_id": "OAB-CYCLE-20260713T194155Z",    "snapshot_ref": "OAB-SNAP-20260713T194200Z"  } }
```

8. Atomic Rollback and Error Handler

8.1 Four Failure Classes

The Atomic Rollback & Error Handler classifies every failure event into one of four classes, each with a defined response protocol. Classification is automated; no operator decision is required to initiate the response for Class I or Class II failures.

Class I — Rail Submission Failure

The transaction is broadcast to the target rail's mempool and rejected — due to insufficient account balance, malformed transaction structure, network congestion, or rail-specific validation failure. The response is immediate re-submission to the primary fallback rail on the Routing Matrix's fallback priority list. The State Vector is updated with a routing amendment record. If all fallback rails also reject, the transaction is advanced to FAILED and the operator is alerted.

Class II — Finality Timeout

The rail accepts the transaction into its mempool, but finality is not confirmed within the configurable timeout window (default: 120 seconds for XRPL and Stellar, 300 seconds for Shibarium at 12-block depth). The response: HTLC expiry is invoked as specified in Paper I, Section 3.2. The escrowed assets are returned to the originator when the timelock expires. The state is advanced to ROLLED_BACK. The transaction is logged in full for operator review and potential re-submission as a new transaction.

Class III — Compliance Breach Mid-Flight

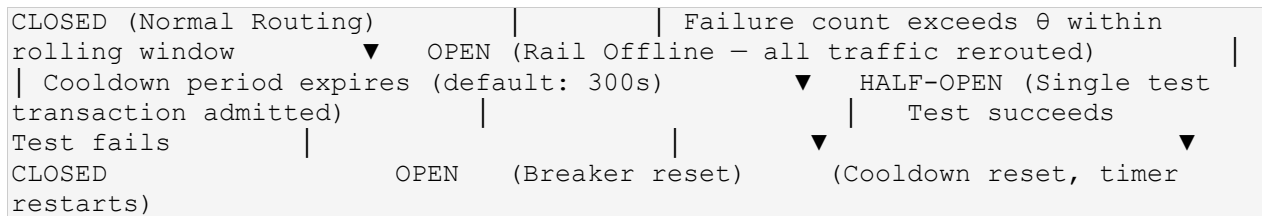
A sanctions list update, ZK attestation expiry, or regulatory freeze is detected after dispatch but before FINALIZED. The CSM signals the Error Handler. The response depends on the current state: if DISPATCHED (not yet confirmed), the HTLC is abandoned. If RAIL_CONFIRMED (confirmed but not CROSS_RAIL_SETTLED), the rail-appropriate clawback mechanism is invoked. The state is advanced to COMPLIANCE_BREACH_ROLLBACK. An operator alert is generated with the specific compliance trigger, the oracle source that detected it, and the actions taken.

Class IV — Oracle Quorum Failure

As described in Section 7: fewer than the quorum minimum of oracle sources are valid for a required data type. The response: halt all new transaction dispatch immediately. In-flight transactions with pinned oracle snapshots and clean compliance state are allowed to finalize. All new incoming instructions from the Compliance Layer are queued with a ORACLE_HOLD status. When oracle quorum is restored, the queue is processed in FIFO order, subject to re-attestation if the hold duration exceeded the ZK proof validity window.

8.2 The Circuit Breaker Pattern

The Circuit Breaker is a per-rail fault isolation mechanism. When Class I failures on a given rail exceed a configurable threshold θ (default: 5 failures within a 60-second rolling window), the Circuit Breaker for that rail is activated — the rail is taken OPEN (offline from routing) for a configurable cooldown period (default: 300 seconds). During the cooldown, all new routing decisions treat the OPEN rail as disqualified. After the cooldown, the Circuit Breaker enters HALF-OPEN state and admits exactly one test transaction. If the test transaction succeeds, the breaker returns to CLOSED (normal operation). If it fails, the breaker returns to OPEN with a reset cooldown timer.



The Circuit Breaker state for each rail is logged in the State Vector Manager's operational ledger and surfaced to the monitoring dashboard in real time. Operators should configure alerting on any OPEN state transition — a rail going OPEN is a signal that the rail itself is experiencing elevated failure rates and warrants direct investigation of the rail's API and network connectivity, independent of the SCE's automated response.

8.3 Rollback Atomicity Guarantee

The SCE guarantees that a rolled-back transaction leaves no partial settlement state on any rail. This guarantee is the hardest operational requirement of the Error Handler. Consider the failure scenario: LEG-001 finalizes on Stellar (asset delivered), but LEG-002 times out on XRPL (cash settlement not confirmed). Without atomicity enforcement, the asset has been delivered but the cash has not been received — a partial settlement that creates an irrecoverable economic loss for the originating party.

The Atomic Rollback Handler's response to this scenario: immediately upon LEG-002 timeout detection, the handler triggers a Stellar clawback for LEG-001, pulling the delivered asset back to the originator's account. The clawback is co-signed by the MPC nodes as defined in Paper I, Section 2.2. Both LEG-001 and LEG-002 are advanced to ROLLED_BACK in the State Vector simultaneously, within a single atomic write operation to the state store. The transaction-level state is advanced to ROLLED_BACK only after both leg-level rollbacks are confirmed.

⚠ Deployment Warning

The atomicity guarantee for cross-rail rollback depends entirely on the Stellar asset having been issued with the `ALLOW_CLAWBACK` flag set. Deploying the SCE with Stellar-settled assets that do not have this flag is an architecture violation that permanently disables cross-rail rollback atomicity for those assets. This must be verified at asset registration time, not at incident time.

9. The Protocol Fee Stream: Integration with the SCE

9.1 Fee Architecture Overview

Paper I, Section 5, defined three revenue models for the multi-rail stack. The Protocol Fee Stream — a percentage fee on transaction volume — is the primary recurring revenue mechanism. This fee does not flow around the SCE. It flows through it. The SCE is not a passive observer of the fee; it is the fee's enforcement and routing mechanism. This design is intentional: by routing the fee through the SCE, the protocol ensures that every transaction that uses the settlement infrastructure contributes to its operating cost, and that fee extraction is logged to the same audit ledger as the settlement event.

9.2 Fee Calculation

The **Fee Calculation Engine** is embedded within the State Vector Manager. At the state transition to `CROSS_RAIL_SETTLED`, the SVM computes the protocol fee as:

$$Fee = V \cdot r_{protocol}$$

Where V is the transaction value in USD and $r_{protocol}$ is the configurable protocol fee rate (default: 0.0001, representing 0.01%). A minimum fee floor of \$0.01 USD applies to all transactions regardless of value, preventing zero-fee micro-transactions from consuming SCE resources. A maximum fee ceiling is configurable per deployment — high-value institutional transactions may negotiate a ceiling cap as part of the white-label licensing agreement.

9.3 Fee Routing and the Protocol Treasury Address

The computed fee is atomically extracted from the transaction settlement at the `CROSS_RAIL_SETTLED` state transition and routed to the **Protocol Treasury Address** — a multi-signature address controlled by the licensing trust's MPC nodes as established in Paper I, Section 2.2. The multi-sig threshold requirement means no single operator node, no single employee, and no single system compromise can redirect the fee stream to an unauthorized address. Fee routing is an MPC-governed operation, not a configuration parameter.

For **white-label licensees** (Paper I, Section 5.1), the SCE supports a configurable fee split at deployment time. The licensee specifies a secondary treasury address and a split ratio (e.g., 70% to licensor, 30% to licensee) in the signed configuration manifest. The SCE routes the computed fee atomically to both addresses in the configured ratio. The split configuration requires MPC-threshold authorization to modify — the licensor's signing nodes must co-sign any change to the licensee's split ratio, preventing a licensee from unilaterally redirecting the full fee to their own treasury.

9.4 Fee Audit Trail

Every fee extraction is recorded as a signed fee record within the transaction's State Vector, referencing the protocol fee transaction hash on the target rail. This creates a continuous, auditable fee ledger that can be reconciled against treasury receipts independently of any off-chain accounting system. The fee

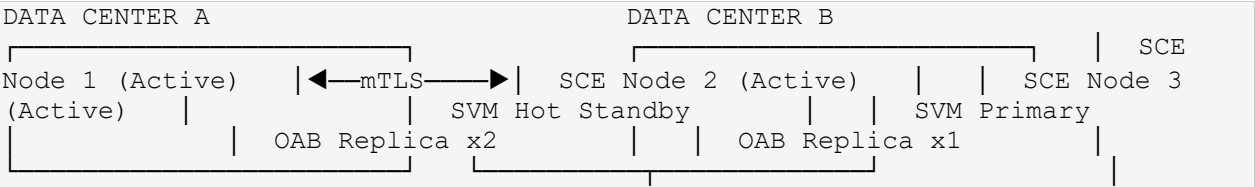
ledger is subject to the same append-only immutability guarantees as the settlement state record — a fee record, once written, cannot be modified or deleted.

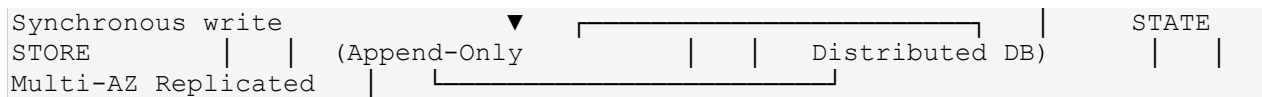
10. Operational Deployment Blueprint

10.1 Distributed Topology Requirements

The SCE is not a single server. A single-server deployment fails every production requirement: it cannot satisfy uptime SLAs for institutional settlement, it cannot tolerate data center failures, and it cannot provide the state replication guarantees required for the append-only ledger. The minimum production deployment topology is as follows:

- 3 SCE nodes in active-active configuration, distributed across at least 2 independent data centers in different failure domains
- 1 dedicated State Vector Manager node (SVM Primary) with synchronous replication to a hot standby SVM in the secondary data center — RPO: zero; RTO: < 30 seconds
- Oracle Arbitration Bus deployed as an independent microservice cluster with a minimum of 3 replicas, at least one per data center
- All inter-node communication encrypted with mutual TLS (mTLS) using certificates issued by an operator-controlled internal CA
- State store: a distributed, append-only database with synchronous multi-node writes (e.g., CockroachDB, YugabyteDB, or equivalent ACID-compliant distributed SQL) — the state store must never accept a write from fewer than a quorum of its replicas





10.2 Infrastructure Requirements Per Node

Component	CPU	RAM	Storage	Network Requirement
SCE Node (Active)	32 vCPU minimum	128 GB	500 GB NVMe SSD	< 10ms to each rail API endpoint
SVM Primary	16 vCPU minimum	64 GB	2 TB NVMe SSD (state growth)	Synchronous replication link < 5ms to standby
SVM Hot Standby	16 vCPU minimum	64 GB	2 TB NVMe SSD	Same as SVM Primary
OAB Replica	8 vCPU minimum	32 GB	250 GB NVMe SSD	Low-latency outbound to all oracle endpoints

Rate limits must be respected for all rail API endpoints. XRPL's public WebSocket API enforces request limits per connection; Stellar Horizon enforces per-IP rate limits. Operators must provision dedicated API access tiers or run their own full node infrastructure for each rail to avoid rate-limit-induced failures at production volume. Implement exponential backoff with jitter for all rail API retries.

10.3 Monitoring and Operational Metrics

Each SCE node emits structured JSON logs to a central log aggregation pipeline. The following six key operational metrics must be monitored with configured alerting thresholds:

- 9. Transaction State Machine Latency (p99):** Time from PENDING_COMPLIANCE to FINALIZED at the 99th percentile. Target: < 30 seconds for Stellar/XRPL primary transactions under normal load.
- 10. Rail-Specific Finality Confirmation Lag:** Per-rail breakdown of the time between DISPATCHED and RAIL_CONFIRMED. Spikes indicate rail-side performance degradation or network congestion.

11. **Oracle Deviation Alert Rate:** The rate at which oracle readings are flagged as anomalous by the OAB. Sustained elevated rates indicate oracle source degradation or a potential manipulation attempt.
12. **Circuit Breaker State Per Rail:** Binary metric — 0 = CLOSED, 1 = OPEN. Any OPEN state should trigger an immediate operator alert and parallel investigation of the affected rail's infrastructure.
13. **Protocol Fee Collection Rate:** USD value of fees collected per hour, compared against expected volume. Significant divergence from expected rate indicates either a volume anomaly or a fee routing malfunction.
14. **Travel Rule ACK Latency (p95):** Time between Travel Rule envelope dispatch and ACK receipt. Target: < 5 seconds. Breaches indicate counterparty VASP gateway issues and may predict upcoming TRAVEL_RULE_PENDING states.

10.4 Configuration Management and MPC Authorization

All SCE operational parameters — fee rates, routing weights, finality thresholds, oracle quorum minimums, circuit breaker thresholds, Travel Rule timeout values, and treasury addresses — are managed through a cryptographically signed **Configuration Manifest**. The manifest is loaded at SCE startup and validated against the operator's MPC signing threshold before any processing begins. A manifest that fails signature validation causes the SCE to refuse to start.

Runtime configuration changes — applied to a running SCE cluster without a restart — require a new manifest signed by the MPC threshold of the operator's signing nodes. No single operator, administrator, or automated system can unilaterally modify the SCE's operational parameters. This design prevents both internal manipulation (rogue operator changing fee rates to zero) and external compromise (an attacker with access to one signing node attempting to redirect the fee stream). The MPC threshold for configuration signing should match the threshold established in Paper I, Section 2.2, for the Trust Layer's signing ceremonies.

11. Integration Checklist for Blueprint Deployers

Engineers and institutions deploying the SCE blueprint must complete the following verification checklist before activating production transaction processing. Each item represents a condition that, if unmet, produces either a compliance failure, a settlement failure, or both. This checklist does not replace load testing, penetration testing, or legal review — it is a baseline technical verification that must be completed first.

11.1 Module 1 — Routing Matrix

- 15. **Asset Profiles Defined:** Every asset class that will be processed by the SCE has a registered Asset Profile Vector with a valid C score, default L multiplier, and T value. No transaction enters the routing pipeline without a pre-registered asset profile.
- 16. **Weight Configuration Calibrated:** The routing weight vector (w_1, w_2, w_3, w_4) has been calibrated for the deployment's specific asset mix and signed into the configuration manifest. Default weights are a starting point, not a production configuration.
- 17. **Fallback Priorities Set:** Each asset profile has a defined fallback rail priority list. No asset profile has Shibarium as a fallback for assets with C score ≥ 6 .
- 18. **Compliance Ceiling Hard Gates Verified:** The Routing Matrix's hard disqualification logic for compliance ceiling breaches has been tested with synthetic transactions that should be disqualified from each rail. Confirm disqualification, not just scoring penalty.

11.2 Module 2 — State Vector Manager

- 19. **State Store Initialized:** The distributed state store is running, multi-replica replication is confirmed active, and a write quorum test has been executed and passed.

- 20. SVM Hot Standby Failover Tested:** SVM Primary failover to hot standby has been tested with a simulated failure. Failover time measured and confirmed within RTO target.
- 21. Finality Thresholds Set Per Rail:** XRPL (1 ledger close), Stellar (1 ledger close), and Shibaumi (≥ 12 blocks, or deployment-specific value) finality thresholds are configured and confirmed in the manifest.
- 22. Append-Only Constraint Verified:** Attempt to delete or modify a state record directly on the state store backend; confirm the operation is rejected by the database's immutability constraint or the SVM's access control layer.

11.3 Module 3 — Compliance Sync Module

- 23. ZK Proof Validity Window Configured:** The proof validity window is set to match the issuing identity provider's proof lifetime. The CSM's validity check has been tested with an expired proof — confirm dispatch is blocked.
- 24. Travel Rule Gateway Endpoints Registered:** All beneficiary VASP gateway endpoints that will be used in production are registered and have been tested with a synthetic Travel Rule envelope. ACK receipt and signature verification confirmed.
- 25. Clawback Flags Verified at Asset Registration:** Every Stellar-settled asset has been verified to have ALLOW_CLAWBACK set on the issuing account. This verification is logged in the asset registry and signed by the deployment operator.
- 26. MPC Clawback Keys Loaded:** The MPC signing nodes required for clawback co-signing are registered, reachable, and have been tested with a synthetic clawback transaction on testnet.

11.4 Module 4 — Oracle Arbitration Bus

- 27. Minimum Three Oracles Per Data Type:** Price, sanctions, jurisdictional, and liquidity oracle sources are each backed by at least three independent providers. Source count per data type logged in the deployment record.

- 28. Anomaly Threshold Configured:** The deviation threshold δ (default: 2.5σ) is confirmed in the configuration manifest. Anomaly flagging has been tested with a synthetic outlier reading.
- 29. Quorum Failure Alerting Tested:** ORACLE_QUORUM_FAILURE has been simulated by disabling oracle sources below quorum threshold on a test environment. Confirm new dispatch is halted and operator alert is generated.
- 30. Sanctions Feed Refresh Verified:** The 15-minute sanctions list refresh cycle is confirmed running. A simulated sanctions list delta update has been processed and applied to the local cache.

11.5 Module 5 — Atomic Rollback Handler

- 31. Circuit Breaker Thresholds Set:** The failure threshold θ (default: 5 failures / 60 seconds) and cooldown period (default: 300 seconds) are confirmed in the configuration manifest for each rail independently.
- 32. HTLC Timeout Values Correct:** HTLC timelock values for all rails are set to exceed the rail's maximum expected finality time by a comfortable margin. XRPL and Stellar: minimum 60 seconds above p99 finality. Shibarium: minimum 10 minutes above expected 12-block confirmation time.
- 33. Cross-Rail Rollback Tested on Testnet:** A deliberate cross-rail partial settlement failure (LEG-001 finalized, LEG-002 timed out) has been executed on testnet. Confirmed: LEG-001 clawback triggered atomically, both legs advanced to ROLLED_BACK simultaneously.
- 34. Rollback Completeness Verified:** Post-rollback, the originator account balance on all involved rails has been verified to match the pre-transaction balance. No partial settlement residue confirmed.

11.6 Module 6 — Protocol Fee Stream

- 35. Treasury Address Set and Multi-Sig Verified:** The Protocol Treasury Address is a valid multi-sig address on the target settlement rail. The required signing threshold has been confirmed with a test withdrawal.
- 36. Fee Rate Configured and Manifest-Signed:** The protocol fee rate r_{protocol} is set in the configuration manifest and the manifest is signed by the required MPC threshold. Direct modification of the fee rate without a new signed manifest has been tested and confirmed to fail.
- 37. Fee Ledger Persistence Verified:** Fee records are confirmed present in the State Vector for all test transactions. Fee ledger can be reconciled against treasury receipts for the test period.
- 38. White-Label Split Configuration (if applicable):** Licensee secondary treasury address and split ratio are confirmed in the manifest. Fee routing to both addresses has been tested with a synthetic transaction.

11.7 Module 7 — Operational Infrastructure

- 39. Monitoring Stack Live:** All six operational metrics defined in Section 10.3 are publishing to the monitoring platform with data points visible. No data gaps in the last 24 hours of pre-production testing.
- 40. Alerting Thresholds Set and Tested:** Each metric has a configured alert threshold. Test alerts have been fired and confirmed received by the on-call rotation.
- 41. mTLS Certificates Installed:** All inter-node mTLS certificates are installed, not expired, and have been verified by a mutual connection test between every node pair in the cluster.
- 42. Configuration Manifest Signed:** The production configuration manifest carries a valid signature meeting the MPC threshold requirement. SCE nodes have been started with this manifest and confirmed to have loaded it successfully in the startup log.
-

12. Conclusion and Paper III Preview

12.1 What the SCE Delivers

The System Connector Engine transforms the three-rail Velocity Layer from three disconnected ledgers into a single, unified settlement machine. Without the SCE, XRPL, Stellar, and Shibarium are exactly what Paper I's opening section diagnosed: silos. Individually capable, collectively fragmented, and architecturally insufficient for institutional multi-asset settlement. The SCE is what makes "multi-rail" mean something operationally real rather than a marketing claim.

The SCE's five sub-modules solve five distinct and non-substitutable problems. The Routing Matrix solves the problem of which rail to use and why, with a documented, auditable answer for every transaction. The State Vector Manager solves the problem of settlement truth — who owns the canonical record of what happened and when. The Compliance Sync Module solves the problem of compliance continuity across rail boundaries and through settlement latency. The Oracle Arbitration Bus solves the problem of data integrity for every decision that depends on external information. The Atomic Rollback Handler solves the problem of partial settlement — the failure mode that, in legacy systems, requires manual reconciliation, legal disputes, and multi-day resolution cycles.

None of these problems disappear if the SCE is not built. They become operational burdens carried by the institutions running the rails directly — which is precisely the operational burden the blueprint universe was designed to eliminate.

12.2 The SCE as the Licensing Keystone

The SCE is not merely infrastructure. It is the licensing mechanism's keystone. Consider what flows through it: the protocol fee stream, which generates operating revenue. The compliance audit trail, which satisfies regulatory reporting obligations. The cross-rail settlement guarantee, which is the commercial value proposition sold to institutional clients. Institutions that build and operate the SCE run

the stack. Those that license it operate on top of it. The SCE is the technical boundary between builder and operator, and that boundary is drawn precisely by the sub-modules defined in this paper.

A white-label licensee running the SCE as delivered — fee routing, compliance enforcement, oracle arbitration, and all — is operationally dependent on the licensor's architecture in every consequential way. The licensor has built the engine. The licensee configures their routes. The configuration manifest, the MPC threshold, and the treasury address are the commercial terms expressed in cryptographic enforcement.

12.3 Paper III Preview

Paper III in the Multi-Rail Fintech Blueprints series is: **"The Trust Layer in Production: MPC Key Ceremony Protocols, State-Vector Anchoring to Bitcoin Taproot, and Cold Storage Governance for Institutional RWA Issuers."**

Paper I established the Trust Layer (Section 2.2) as the MPC-backed cryptographic foundation beneath the entire multi-rail stack — the layer that co-signs clawbacks, authorizes configuration changes, and anchors state vectors for long-term auditability. Paper II has referenced that layer repeatedly, in the context of fee routing, configuration manifest signing, and clawback coordination. Paper III will engineer that layer with the same depth this paper applied to the SCE.

Paper III will cover: the threshold cryptography operational procedures for production MPC deployments; the air-gapped signing ceremony design for key issuance and rotation; the construction of Bitcoin Taproot anchoring transactions for state vector commitments; and the institutional governance policies — board authorization, multi-party approval workflows, key holder responsibilities — that must sit above the cryptographic layer to produce a governance structure that satisfies institutional audit requirements. The cryptography alone is necessary but not sufficient. The governance that operates it is what makes it auditable, defensible, and institutionally credible.

Glossary

Atomic Rollback

A rollback operation that guarantees all legs of a multi-rail transaction are reversed simultaneously and completely, with no partial settlement state persisting on any rail after the operation completes.

Asset Profile Vector (P^*)

A five-component vector (C, V, J, L, T) characterizing an asset transfer instruction for the purpose of Routing Matrix computation. Components represent: Asset Class Score, Transaction Value, Jurisdictional Complexity, Required Liquidity Depth, and Throughput Priority.

Circuit Breaker

A per-rail fault isolation mechanism within the Atomic Rollback & Error Handler. When a rail's failure rate exceeds a configurable threshold, the breaker opens, removing that rail from routing for a cooldown period before re-admitting test traffic in HALF-OPEN state.

COMPLIANCE_BREACH_ROLLBACK

A terminal error state in the SCE state machine indicating that a compliance event (sanctions hit, attestation expiry, regulatory freeze) was detected mid-flight and all transaction legs have been rolled back through the appropriate rail-specific mechanism.

Compliance Sync Module (CSM)

The SCE sub-module responsible for enforcing compliance continuity across the settlement lifecycle: validating ZK attestation windows, dispatching and managing Travel Rule envelopes, and coordinating clawback or HTLC abandonment on compliance breach events.

Configuration Manifest

A cryptographically signed document containing all operational parameters for an SCE deployment. Loaded at startup and validated against the operator's MPC signing threshold. All runtime configuration changes require a new manifest signed by the required MPC threshold.

Finality Monitor

A per-rail polling process within the State Vector Manager that monitors the confirmation and finality status of dispatched transactions. Triggers state advancement from DISPATCHED to RAIL_CONFIRMED upon finality detection.

HTLC (Hashed Timelock Contract)

A cryptographic construct that escrows assets conditionally: release requires presentation of a hash preimage within a time limit. Used by the SCE for atomic cross-rail settlement on XRPL and Shibarium, and as the primary rollback mechanism for Class II (Finality Timeout) failures.

Oracle Arbitration Bus (OAB)

The SCE sub-module that aggregates external data feeds (price, sanctions, jurisdictional rules, liquidity depth) from multiple independent sources and applies weighted median arbitration to produce anomaly-resistant aggregated values for consumption by other SCE sub-modules.

Protocol Treasury Address

The multi-signature address controlled by the licensing trust's MPC nodes to which protocol fees are routed. No single party can redirect the fee stream without meeting the MPC signing threshold.

Rail Fitness Score (F)

A composite score computed by the Routing Matrix for each candidate rail, combining weighted sub-scores for compliance suitability, liquidity availability, throughput match, and cost. The highest-scoring non-disqualified rail receives the transaction.

Routing Matrix

The SCE sub-module that receives an incoming asset transfer instruction and computes Rail Fitness Scores for each candidate rail, selecting the optimal rail and producing an ordered fallback priority list. Routing decisions are fully logged to the State Vector Manager.

State Vector

The canonical, append-only record maintained by the State Vector Manager for a single transaction. Contains the full state history, per-leg settlement status, compliance attestation hash, Travel Rule ACK reference, and protocol fee record.

State Vector Manager (SVM)

The SCE sub-module and sole writer to the canonical state store. Maintains the cross-rail state ledger, runs per-rail Finality Monitors, calculates protocol fees at CROSS_RAIL_SETTLED transitions, and produces the audit lineage for every transaction the SCE processes.

System Connector Engine (SCE)

The active, stateful middleware layer in the multi-rail RWA stack that sits between the Compliance/Identity Layer and the Velocity Layer (XRPL, Stellar, Shibarium). The SCE owns the canonical settlement record, enforces compliance continuity, routes transactions to optimal rails, arbitrates oracle data, and executes atomic rollbacks on failure.

Travel Rule Envelope

The off-chain data package required by FATF Recommendation 16 for transactions exceeding the statutory threshold. Assembled and dispatched by the CSM simultaneously with on-chain transaction broadcast. Contains encrypted originator and beneficiary information, ZK attestation hash reference, and a cryptographic ACK slot.

Weighted Median Arbitration

The aggregation algorithm applied by the Oracle Arbitration Bus to multi-source oracle readings. Computes the confidence-weighted median of available readings, providing manipulation resistance superior to weighted mean aggregation by being robust to single-source extreme outliers.

ZK Attestation Hash

A SHA-512 hash of the ZK proof issued by the Compliance/Identity Layer's identity provider, stored in the transaction's State Vector at PENDING_COMPLIANCE. Used by the CSM to verify compliance continuity without re-transmitting the proof itself across settlement legs.

Technical White Paper

|

Paper II

|

Leon Calvin Long II

|

July 13, 2026

|

Confidential